

We cannot (realistically) get rid of throwing moves

Abstract

During the reflector discussion about variant, there were suggestions to restrict variant to types with non-throwing move assignments, and suggestions to ban throwing move assignments in general, and perhaps also throwing move constructors. This paper provides some thoughts on those ideas.

The suggested outcome is that we can't realistically get rid of throwing move operations because they end up being more common than people realize, easier to end up with than people realize, and when it comes to variant, banning types that have throwing moves will significantly and severely limit the set of user-defined types that can be used in a variant. The recommendation of this paper is to not pursue such directions.

Background material

The following papers provide useful background material:

[N2983, Allowing Move Constructors to Throw](#)

[N3401, Generating move operations \(elaborating on Core 1402\)](#)

What's a "throwing move"?

For constructors, a "throwing move" is not just a move constructor that is not noexcept, it's **anything that causes the expression**

```
X(move(rhs))
```

to be potentially throwing (aka not noexcept(true)), **including but not limited to the move constructor of X**. In particular, **if overload resolution ends up selecting a copy constructor to be used in that expression**, the expression (very often, but not always - your type might have a non-throwing copy constructor) is a "throwing move".

For assignment, a "throwing move" is not just a move assignment operator that is not noexcept, it's **anything that causes the expression**

```
lhs = move(rhs)
```

to be potentially throwing (aka not noexcept(true)), **including but not limited to the move assignment operator of X**. In particular, **if overload resolution ends up selecting a copy assignment operator to be used in that expression**, the expression (very often, but not always - your type might have a noexcept copy assignment operator) is a "throwing move".

Is it common?

Let's look at a very simple example (credits to Jonathan Wakely):

```
struct X
{
    // other value-initializing constructors left out
    X(const X&);
};
```

This type has its move operations suppressed due to a user-declared copy operation, but it can still be used in either of the expressions we showed before when describing what is a "throwing move". It has a "throwing move".

Here's another simple example:

```
struct X2
{
    ~X2();
    // data members not shown, let's please assume they are
```

```
// either copyable or movable
};
```

This type has its move operations suppressed due to a user-declared destructor, but it can still be used in either of the expressions we showed before when describing what is a "throwing move". It may or may not have a throwing move, depending on what its data members are. When this type is used in the move expressions, the move operations of the members of X2 will not be used, so it boils down to whether the members of X2 have copy operations that are non-throwing. This means that the types of the members need not be like X, all it takes is that one of them has a **throwing copy operation, regardless of whether it also has a non-throwing move operation**.

So, are such types common? You bet they are. I have written dozens and dozens of them, and not all of them are necessarily in legacy code. Some of that code may be new, but needs to work in pre-C++11 environments as well. My customers and users have thousands of such types. Yours have more.

So can we ban throwing moves?

Ban them where?

- In the Core language? If we want to create a massively incompatible fork of C++ below the standard version that enforces the ban, then maybe.
- In the Standard Library? If we want to create a little less massively but still massively incompatible fork of C++ below the standard version that enforces the ban, then maybe.
- In the proposed `std::variant`? Maybe, but then we rule out all the aforementioned types from being used in a variant.

Let me take off the "calm and cool paper writer hat" for a second. I'll write those bullet answers from a personal perspective, and what I'd likely suggest the national perspective to be:

- In the Core language? That sounds like a fairly.. ..dogmatic approach, to enforce design so forcefully at that level, and I have doubts about whether it would have the desired effect. A more likely outcome is that people who have existing code will decide to not adopt a new C++ standard, or will request from their vendor that the vendor doesn't follow it in this regard.
- In the Standard Library? Same answer.
- In the proposed `std::variant`? Why is variant special? We already have other less drastic options available for it, so banning all these types from being used in variant is hardly explicable, when they can be used in other library wrappers and containers without huge trouble. (Yes, I can imagine what we'll explain among ourselves to be the reason. My users will not accept that explanation, especially not when `boost::variant` doesn't have such a restriction).

Wait a minute. We could just copy such types, instead of moving them, using `move_if_noexcept`, right?

Let's look at another example:

```
pair<X, unique_ptr<Foo>>
```

This type is not copyable, because it has a move-only member. So, getting back to the papers mentioned in the beginning, we could either choose that it has a throwing move, or it's not copyable nor movable. If it's not copyable nor movable, I don't think I can store it in a vector, unless we do a lot of surgery on vector. Similar problems arise with other containers. If we look outside the standard library, I can no longer return such a type by value nor pass it by value. It becomes a scoped type that will not transfer outside its scope.

Howard Hinnant pointed out that there's a perhaps surprising example of a type that otherwise would be nicely non-throw-movable, but isn't when it's suitably qualified:

```
const vector<int>
```

That type has a throwing move.

Let's look at yet another example:

```
pair<X, vector<Foo>>
```

So, perhaps this type wouldn't have move operations, and we could happily copy it around. The problem is that one major reason why throwing moves were allowed was that we would like to be able to move the vector even in cases where we can't move the X. If we never move such a type, we will then always copy it, which can never move the vector.

But some of those types can be made movable. Can't we expect that all copyable

types will be made movable?

Sadly, no. Tomasz Kaminski depicted a type like the following:

```
class Employee
{
    string name;
    string surname;
public:
    Employee(const Employee&); // the type is copyable, moves not generated
    // the invariant of the type includes that the members aren't empty
    // initializing constructors and the rest of the interface left out
    // for brevity
};
```

The business-domain rules for such a type require that its members are not empty. Ever. So, how do we implement a non-throwing move for it? We can't move the strings. We can copy them, and we're back at a throwing move that we already had. Similar invariants were mentioned as a concern for generated move operations in general in [N3153, Implicit Move Must Go](#). We decided that the suppression rules we have are sufficiently non-breaking, and the Employee class plays by those rules - it protects its invariant even in C++11, and there's no generated move operation that would break its invariant. Nor can one be reasonably written, so requiring such a move, especially a non-throwing one, seems unattainable.

So, how incompatible would the ban of throwing moves be?

Any type like X written before or while C++11 and C++14 are used would become incompatible with the standard that enacts the ban. Same goes for the types with a mixture of copy-only and move-only members. The period "while C++11 and C++14 are used" may take longer than we think. Sure, the example type that uses the aforementioned mixture can appear only after C++11 is taken into use. But any type like X can still be used with C++11 and C++14, and any such code would break when the ban is enacted.

I can't support a wide-effect ban. I must oppose it, as hard as I can. For variant, I think variant becomes a much less useful type if it has such a variant-specific ban. It becomes very hard to justify using the standard variant instead of boost::variant, and that removes a large part of the very good reasons why we standardize libraries. (There's nothing wrong with using boost, but there are good reasons to use a standard type instead of a boost type, if possible.)